



UCCD2303 DATABASE TECHNOLOGY / UCCD2203 DATABASE SYSTEMS

Introduction to Data Modelling and Design Part 2

Lecture 2

By

Mr. Cheang Kah Wai, Alvin

Department of Information Systems, FICT

Lecture 2 Topic: Introduction to Data Modelling and Design - Part 2

- Enhanced Entity Relationship (EER) Modelling
- The Relational Model

ENHANCED ENTITY RELATIONSHIP (EER) MODELLING

Enhanced Entity Relationship (EER) Model

- Complexity of data structures being modeled has increased and application software requirements have become more stringent.
- The need to capture more information in the data model has increased.
- Extension to Entity Relationship (ER) Model.
- Adding more semantic constructs to the original ER model.

EER Model Entity Supertypes and Subtypes

- Most employees possess a wide range of skills and special qualifications.
- Data modelers must find a variety of ways to group employees based on their characteristics.
- Example:
 - A retail company can group employees as salaried and hourly
 - A university could group employees as faculty, staff, and administrators.

EER Model Entity Supertypes and Subtypes

- The benefits of grouping of employees into various types:
 - Avoid unnecessary nulls in attributes when some employees have characteristics that are not shared by other employees.
 - Enables a particular employee type to participate in relationships that are unique to that employee type.

EER Model Entity Supertypes and Subtypes

- Let's explore an aviation business that employs pilots, mechanics, secretaries, accountants, database managers, and many other types of employees.
 - Pilots share certain characteristics with other employees, such as, last name and hire date.
 - Nevertheless, many pilot characteristics are not shared by other employees.
 - Pilots must meet special requirements such as flight hour restrictions, flight checks and periodic training.

EER Model Entity Supertypes and Subtypes

- Therefore, if all employee characteristics and special qualifications were stored in a single EMPLOYEE entity, you will have a lot of nulls, or you will have to create needless dummy entries.
 - For example, employee license and employee ratings are only applicable (special characteristics) to pilots
 - This will create null values for employees who are not pilot.

EER Model

Entity Supertypes and Subtypes

- Pilots participate in some relationships that are unique to their qualifications.
 - For example, not all employees can fly airplanes; only employees who are pilots can participate in the “*employee flies airplane*” relationship.

EER Model Entity Supertypes and Subtypes

- PILOT entity stores only attributes that are unique to pilots and EMPLOYEE entity stores attributes that are common to all employees.
 - ❑ PILOT is a *subtype* of EMPLOYEE.
 - ❑ EMPLOYEE is a *supertype* of PILOT.
- In modeling terms, an **entity supertype is a generic entity type that is related to one or more entity subtypes**.
 - ❑ The entity supertype contains common characteristics.
 - ❑ Each of the entity subtype contain their own unique characteristics.

Note: Supertypes and subtypes also known as superclasses and subclasses.

EER Model

Entity Supertypes and Subtypes

- Two criteria to help the database designer determines when to use subtypes and supertypes:
 - There must be different, identifiable kinds or types of the entity in the user's environment.
 - The different kinds or types of instances should each have one or more attributes that are unique to that kind or type of instance.

EER Model

Specialization

- Process of defining a set of subtypes of an entity type (top-down).
 - Supertypes - Parent entities
 - Subtypes – Child entities
- Specialization hierarchy reflects the 1:1 relationship between the supertype and its child entities.
 - For example, a PILOT subtype occurrence is related to one instance of the EMPLOYEE supertype.

EER Model

Specialization

- The relationships depicted within the specialization hierarchy are sometimes described in terms of “is-a” relationships.
- For example, a pilot is an employee, a mechanic is an employee, and an accountant is an employee.

EER Model

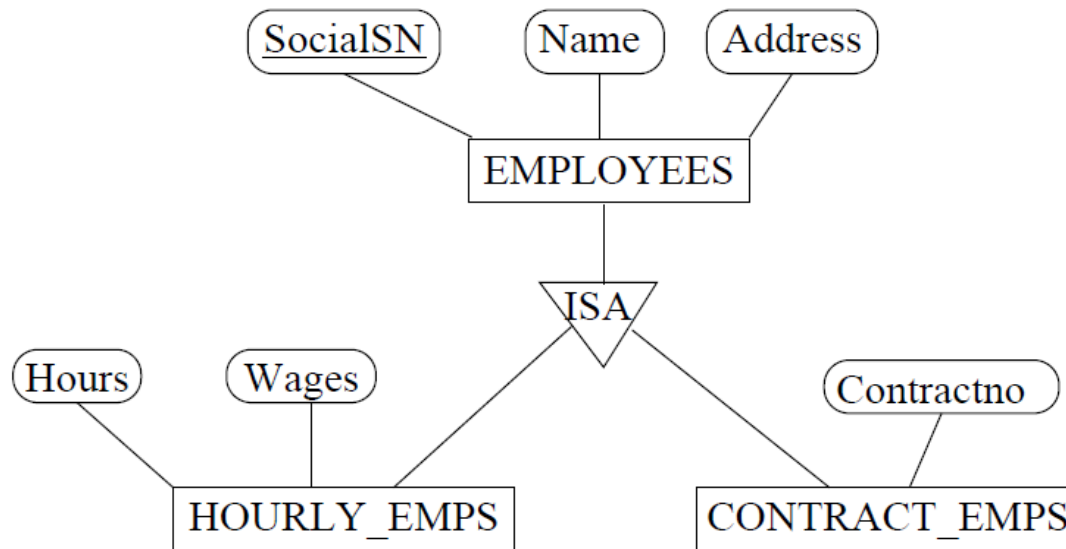
Specialization

■ Important to note:

- ❑ A subtype can exist only within the context of a supertype, and every subtype can have only one supertype to which it is directly related.
- ❑ However, can have many levels of supertype or subtype relationships – you can have a specialization hierarchy in which a supertype has many subtypes. In turn, one of the subtypes is the supertype to other lower-level subtypes.

EER Model

Specialization



- **HOURLY_EMPS** is a subtype of **EMPLOYEES** and thus inherits its attributes and relationships.
- **CONTRACT_EMPS** is a subtype of **EMPLOYEES** and thus inherits its attributes and relationships.

EER Model

Specialization

- Specialization hierarchies enable the data model to capture additional semantic content or meaning into the ERD.
 - Support attribute inheritance.
 - Define a special supertype attribute known as the subtype discriminator.
 - Define disjoint or overlapping constraints and complete or partial constraints.

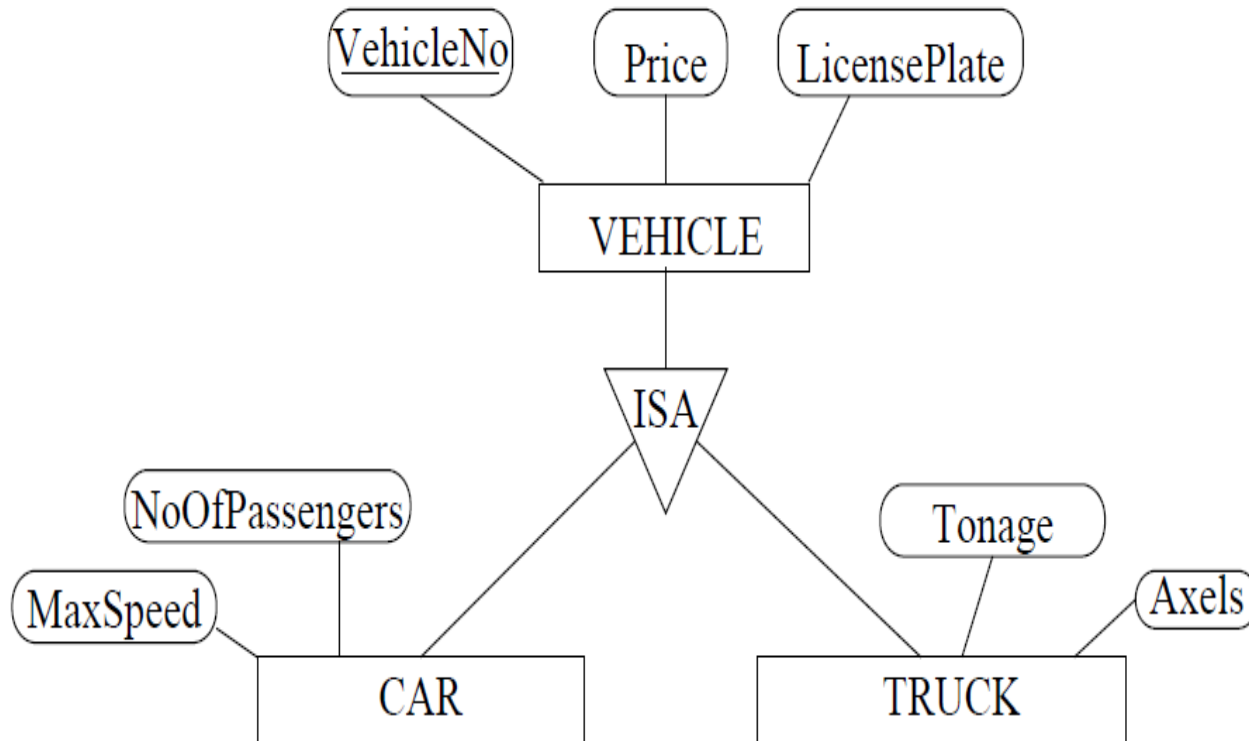
EER Model

Generalization

- Bottom-up process of identifying a higher-level, more generic entity supertype from lower-level entity subtypes.
- Based on grouping the common characteristics and relationships of the subtypes.
 - For example, you might identify multiple types of musical instruments: piano, violin, and guitar. Using the generalization approach, you could identify a “string instrument” entity supertype to hold the common characteristics of the multiple subtypes.

EER Model

Generalization



EER Model

Inheritance

- The property of inheritance enables an entity subtype to inherit the attributes and relationships of the supertype.
- One important inheritance characteristic is that all entity subtypes inherit their primary key attribute from their supertype.
- Subtypes can have additional attributes and relationships.

EER Model

Subtype Discriminator

- A **subtype discriminator** is the attribute in the supertype entity that determines to which subtype the supertype occurrence is related.

EER Model

Disjoint and Overlapping Constraints

- **Disjoint subtypes** (also known as **non-overlapping** subtypes) are subtypes that contain a unique subset of the supertype entity set.
 - For example, an employee (supertype) who is a **pilot** (subtype) can appear only in the **PILOT** subtype, not in any of the other subtypes.
 - In an ERD, disjoint types are indicated by the **letter *d*** inside the category shape.

EER Model

Disjoint and Overlapping Constraints

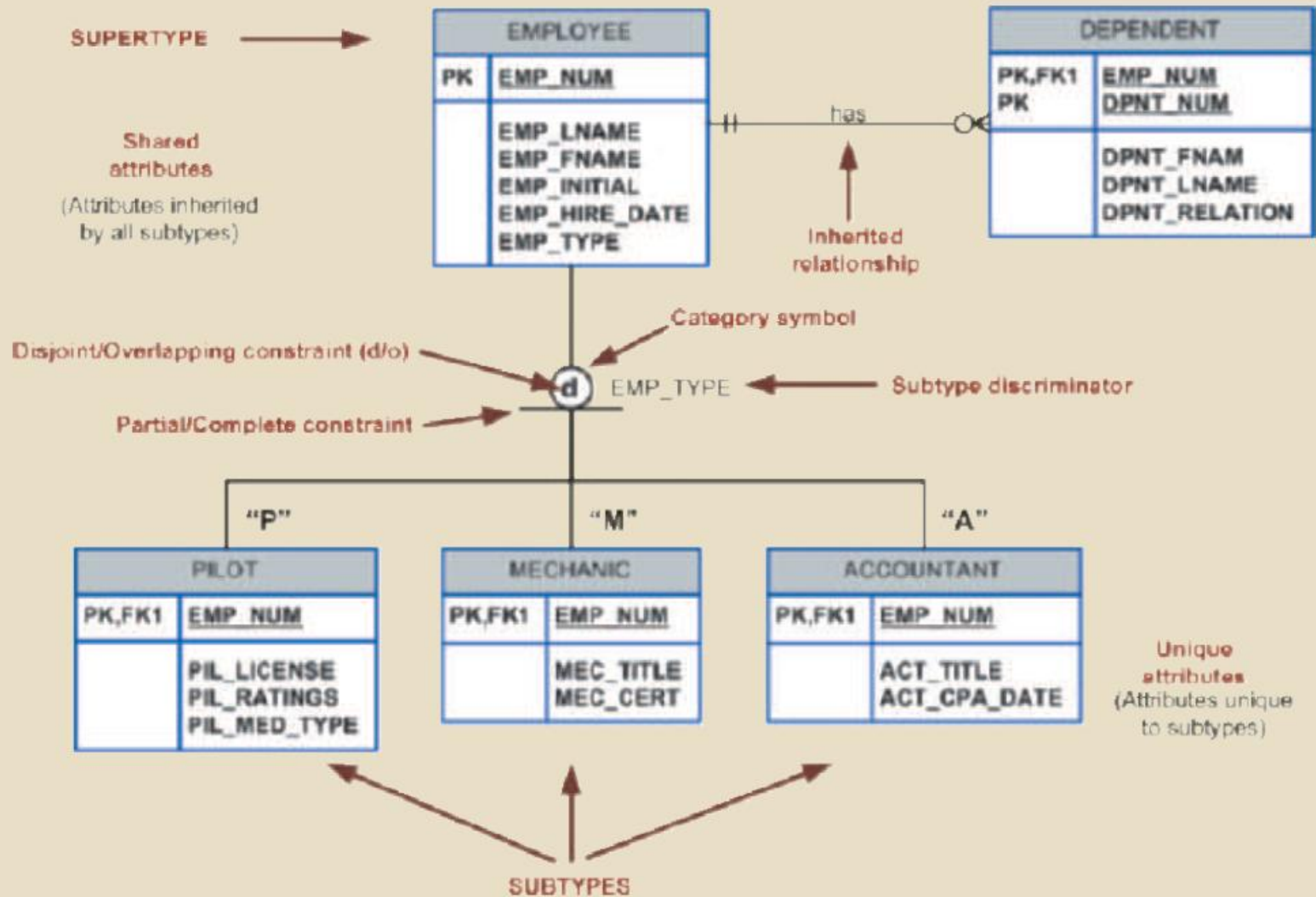
- **Overlapping subtypes** are subtypes that contain non-unique subsets of the supertype entity set; that is, each entity instance of the supertype may appear in more than one subtype.
 - ❑ For example, in a university environment, a person may be an employee, a student, or both.
 - ❑ In turn, an employee may be a professor as well as an administrator.
 - ❑ **STUDENT** and **EMPLOYEE** are overlapping subtypes of the supertype **PERSON**.
 - ❑ **PROFESSOR** and **ADMINISTRATOR** are overlapping subtypes of the supertype **EMPLOYEE**.
 - ❑ In an ERD, overlapping subtypes are indicated by the letter *o* inside the category shape.

EER Model

Completeness Constraints

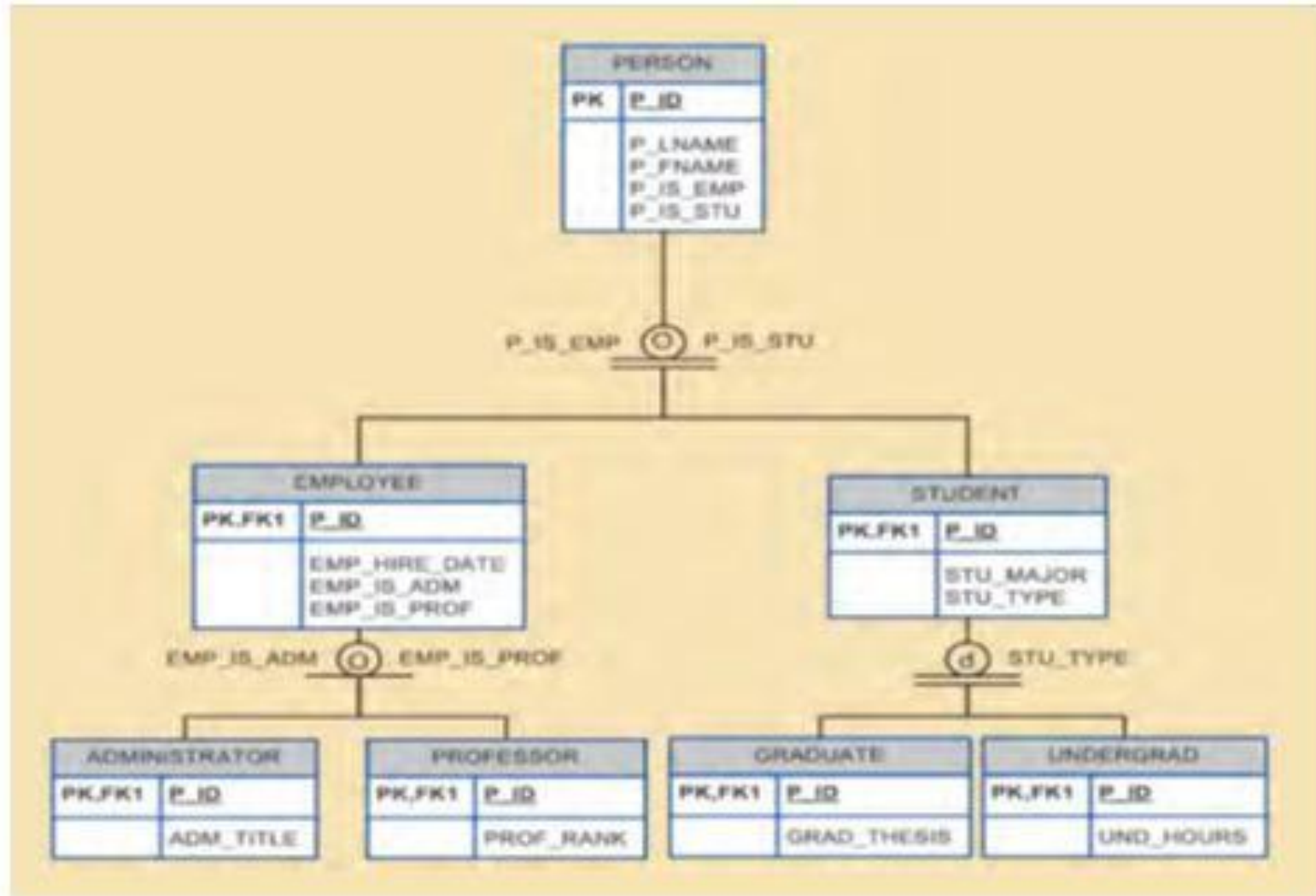
- The **completeness constraint** specifies whether each entity supertype occurrence must also be a member of **at least one subtype** – can be partial or total.
- **Partial completeness** means that not every supertype occurrence is a member of a subtype; some supertype occurrences may not be members of any subtype.
- **Total completeness** means that every supertype occurrence **must be a member** of at least one subtype.

EER Model



Source: Coronel & Morris, Database Systems: Design, Implementation, & Management, 13e.

EER Model



Source: Coronel & Morris, Database Systems: Design, Implementation, & Management, 13e.

EER Model

TYPE	DISJOINT CONSTRAINT	OVERLAPPING CONSTRAINT
Partial 	Supertype has optional subtypes. Subtype discriminator can be null. Subtype sets are unique.	Supertype has optional subtypes. Subtype discriminators can be null. Subtype sets are not unique.
Total 	Every supertype occurrence is a member of a (at least one) subtype. Subtype discriminator cannot be null. Subtype sets are unique.	Every supertype occurrence is a member of a (at least one) subtype. Subtype discriminators cannot be null. Subtype sets are not unique.

THE RELATIONAL MODEL

The origins of the Relational Model

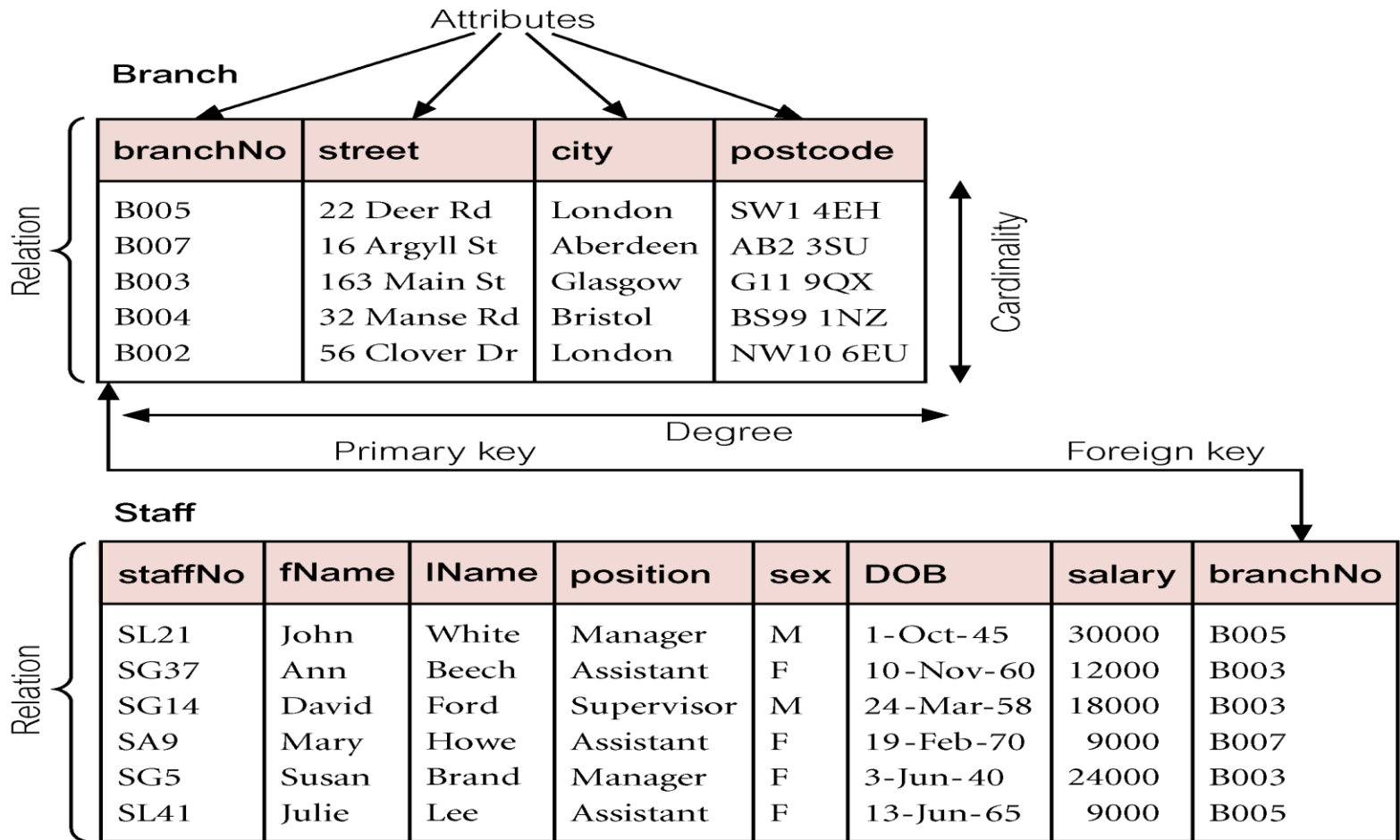
- Proposed by E.F.Codd (1970)
- Pioneer projects such as at IBM's relational DBMS System **R**, and UC-Berkeley's relational Model **INGRES** (in mid-1970s)
- Today, still the dominant database model:
 - IBM DB2, ORACLE, INFORMIX, SYBASE
 - MICROSOFT Access, SQL Server
 - FOXBASE, PARADOX
 - Still more....
- In the relational model, all data is logically structured within relations (tables).

Terminologies used in Relational Model

Relational Model is based on the mathematical concept of a **relation** (physically represented as **Table**).

- **Relation:** A Relation is a table with *Rows* and *Columns*.
- **Attribute:** An Attribute is a *named column* of a relation.
- **Domain:** A Domain is the set of *allowable values for one or more attributes*.
- **Tuple:** A Tuple is a *row of a relation*.
- **Degree:** The Degree of a Relation is *the number of attributes it contains*.
Unary relation, binary relation, ternary & n-ary relations
- **Cardinality:** The Cardinality of a relation is *the number of tuples it contains*.
- **Relational Database:** A collection of normalized relations with distinct relation names.

Instances of the Branch and Staff relations



Refer: Fig4.1, Page: 145

Example of Attribute Domains

Attribute	Domain Name	Meaning	Domain Definition
branchNo	BranchNumbers	The set of all possible branch numbers	character: size 4, range B001–B999
street	StreetNames	The set of all street names in Britain	character: size 25
city	CityNames	The set of all city names in Britain	character: size 15
postcode	Postcodes	The set of all postcodes in Britain	character: size 8
sex	Sex	The sex of a person	character: size 1, value M or F
DOB	DatesOfBirth	Possible values of staff birth dates	date, range from 1-Jan-20, format dd-mmm-yy
salary	Salaries	Possible values of staff salaries	monetary: 7 digits, range 6000.00–40000.00

Refer: Fig 4.2, Page: 145

Relational Model Terms and their SQL Equivalents

SQL term	Relational database term	Description
<i>Row</i>	<i>Tuple</i> or <i>record</i>	A data set representing a single item
<i>Column</i>	<i>Attribute</i> or <i>field</i>	A labeled element of a tuple, e.g. "Address" or "Date of birth"
<i>Table</i>	<i>Relation</i> or <i>Base relvar</i>	A set of tuples sharing the same attributes; a set of columns and rows
<i>View</i> or <i>result set</i>	<i>Derived relvar</i>	Any set of tuples; a data report from the RDBMS in response to a query

Formal terms	Alternative 1	Alternative 2
Relation	Table	File
Tuple	Row	Record
Attribute	Column	Field

Refer: Table 4.1, Page: 146

Attribute domain can be shared

```
CREATE DOMAIN street AS VARCHAR2(30),
```

```
CREATE TABLE student(  
StNo    number(6) NOT NULL,
```

```
...  
StStreet street,  
....);
```

```
CREATE TABLE faculty(  
FaNo    number(4) NOT NULL,
```

```
...  
FaStreet street,  
....);
```

Database relations

■ What is Schema?

- It is the structure of a database system, described in a formal language supported by the Database Management System (DBMS).
- In a relational database, the schema defines the tables, the fields in each table, and the relationships between fields and tables.
- Schemas are generally stored in a data dictionary.
- Although a schema is defined in text database language, the term is often used to refer to a graphical depiction of the database structure.

■ Relation schema:

- Named relation defined by a set of attribute and domain name pairs.

$S1 = \{\text{branchno:B005, street:22, city:London}\}$

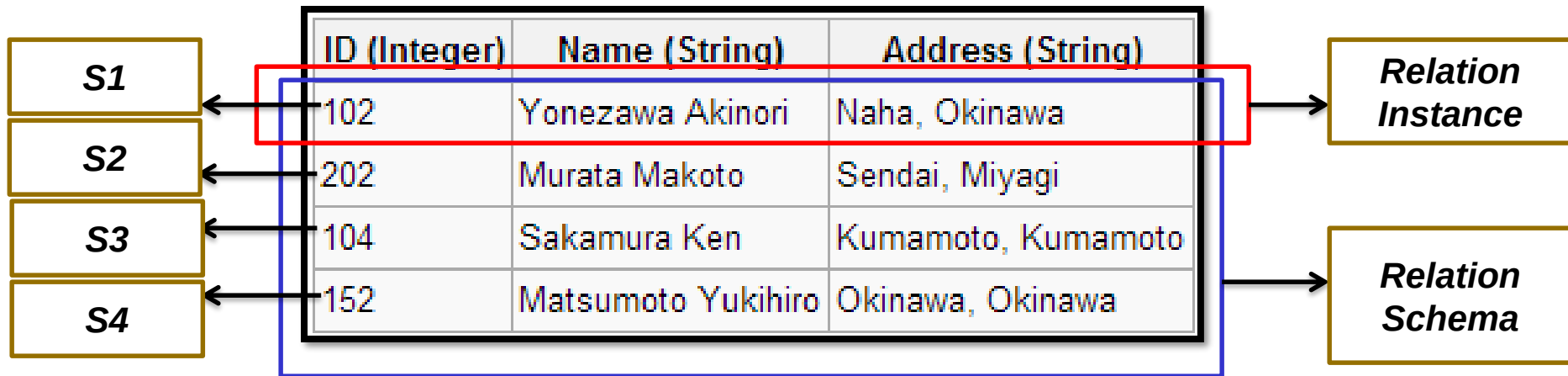
$R1 = \{S_1, S_2, \dots, S_n\}$

■ Relational database schema:

- Set of relation schemas, each with a distinct name.

$R = \{R_1, R_2, R_3, \dots, R_n\}$

Example of a Relation



Relation Schema

$$R1 = \{S_1, S_2, S_3, \dots, S_n\}$$

Relational Database Schema

$$R = \{R_1, R_2, R_3, \dots, R_n\}$$

Relational Database Schema – Textual Form (DreamHome Rental)

Branch	(<u>branchNo</u> , street, city, postcode)
Staff	(<u>staffNo</u> , fName, lName, position, sex, DOB, salary, branchNo)
PropertyForRent	(<u>propertyNo</u> , street, city, postcode, type, rooms, rent, ownerNo, staffNo, branchNo)
Client	(<u>clientNo</u> , fName, lName, telNo, prefType, maxRent)
PrivateOwner	(<u>ownerNo</u> , fName, lName, address, telNo)
Viewing	(<u>clientNo</u> , <u>propertyNo</u> , viewDate, comment)
Registration	(<u>clientNo</u> , <u>branchNo</u> , staffNo, dateJoined)

underlined attributes are Primary keys.

Usually asterisk marked (*) attributes are foreign keys, e.g. *staffNo.

Relational Database Schema is also simply known as Relational Schema

Relational Database Schema – Graphical Form (DreamHome Rental)

Branch				Staff							
branchNo	street	city	postcode	staffNo	fName	lName	position	sex	DOB	salary	branchNo
B005	22 Deer Rd	London	SW1 4EH	SL21	John	White	Manager	M	1-Oct-45	30000	B005
B007	16 Argyll St	Aberdeen	AB2 3SU	SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
B003	163 Main St	Glasgow	G11 9QX	SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
B004	32 Manse Rd	Bristol	BS99 1NZ	SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
B002	56 Clover Dr	London	NW10 6EU	SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
				SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

PropertyForRent									
propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650	CO46	SA9	B007
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87	SL41	B005
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40		B003
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93	SG37	B003
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87	SG37	B003
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93	SG14	B003

Properties of Relations

- Relation name is distinct from all other relation names in relational schema.
- Each cell of relation contains exactly one atomic (single) value.
- Each attribute has a distinct name.
- Values of an attribute are all from the same domain.
- Each tuple is distinct; there are no duplicate tuples.
- Order of attributes and of tuples has no significance.

Relational Keys

■ Superkey

An attribute, or a set of attributes, that uniquely identifies a tuple within a relation.

May contain additional attributes not necessarily be unique. e.g.
Icno, stid, (stname,major), (stid,stname)

■ Candidate Key

A minimal superkey. A superkey is minimal if removing any columns makes it no longer unique.

The candidate key has two properties: uniqueness and irreducibility.

Relational Keys (contd..)

■ **Primary Key:**

The candidate key that is selected to identify tuples uniquely within relation.

Every record must have a primary key and primary key **cannot be NULL**.

An attribute (**or combination of attributes**) that uniquely identifies any given row.

Alternate Key/Secondary Key: The candidate keys that are not selected as primary key.

■ **Foreign Key:**

An attribute, or set of attributes, within one relation that matches the candidate key of some (**possibly the same**) relation.

Foreign key creates a relationship between the two tables.

Foreign key value must exist in the table where referred to.

Relational Keys (contd..)

■ Surrogate Key:

- Created to be the record's primary key identifier when no suitable primary key exists
- Surrogate key has no real relationship to the record to which it is assigned, other than to identify the record uniquely
- Developers configure the database to generate surrogate key values automatically
- Surrogate keys are always numerical fields, because the database generates surrogate key values automatically by incrementing the previous value by one

Example for Relational keys

Name	Age	SSN	Phone Extension
Rob	28	12346789	1242
Amy	34	59889879	1333
Elizabeth	34	56887744	9312
Jim	42	45558899	9899
Mike	29	55256665	9989

Examples: There are many superkeys in the sample table above.

For example, (SSN), (SSN, Phone Extension, Name) and (SSN, Name), etc. Of those listed, only <SSN> is a candidate key, as the others contain information not necessary to uniquely identify records.

Superkey = {SSN, Ph. Ext., (SSN, Ph. Ext.), (SSN, Age), (SSN, Name), (SSN, Age, Name), (SSN, Ph. Ext., Age, Name), }

Composite Key = {(SSN, Ph. Ext.), (SSN, Age, Name),}

Candidate Key = {SSN, Ph. Ext.}

Primary Key = {SSN}

Alternate Key = {Ph. Ext.}

**Assume one staff is only assigned to one Telephone Extension and there are two staffs with the same name.*

Example for Relational keys

Example: Consider the following tables:

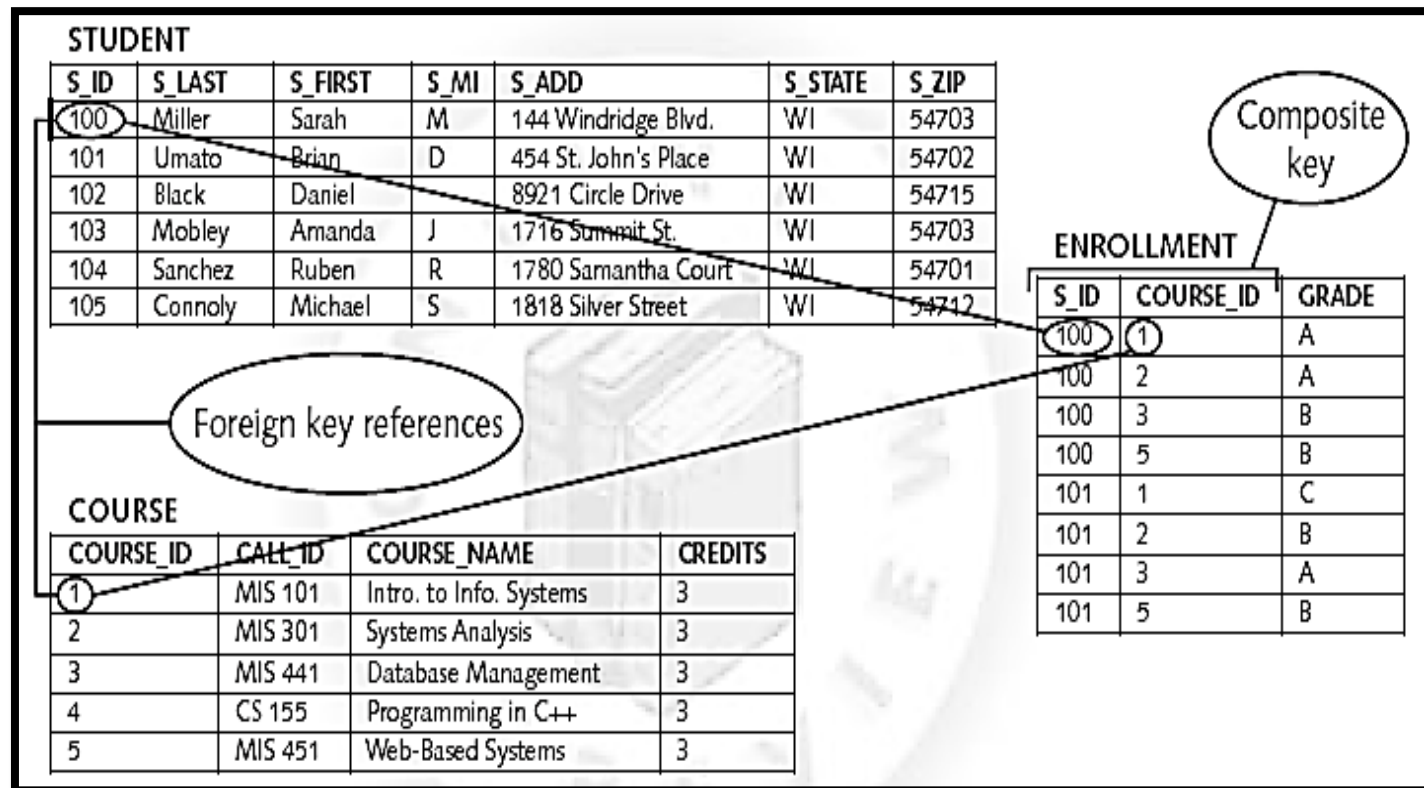
Staff(staffNo, SSN, name, position, salary, branchNo)

Branch(branchNo,street,city,state,zipCode, mgrStaffNo)

- **SuperKey** = {staffNo, (staffNo, name), (staffNo, position), (staffNo, SSN), (staffNo, branchNo), (staffNo, name, position), (staffNo, name, salary), (SSN,name), ... }
- **Candidate Key** = {staffNo, SSN}
- **Primary Key** = {staffNo}
- **Foreign Key** = {branchNo, mgrStaffNo}

Composite Key

- is a unique key that you create by combining two or more fields
- is usually comprised of fields that are primary keys in other tables



Composite key (S_ID and COURSE_ID combined). The value combinations must be unique.

Constraints

Rules that restrict the data values that you can enter into a field in a database table.

- Types of constraint:

- **Integrity constraints:** Define primary and foreign keys.

- **Entity Integrity:** In a base relation, no attribute of a primary key can be null. No two rows with the same **primary key** value

- **Referential Integrity:** If foreign key exists in a relation, either foreign key value must match a candidate key value of some tuple in its home relation or **foreign key** value must be wholly null.

- **Value/Field/Domain Constraint:** limits the value that can be placed in a specific field, irrespective of values that exist in other table records.

Constraints (contd...)

- ❑ **General constraints:** Additional rule specified by users or database administrators of a database that define or constrain some aspect of the enterprise.

E.g. Only 20 workers can work in a Branch or a convent school only accept female students.

Constraints (contd...)

The following are the **Value Constraints**:

- **Check constraints** : field value must be a specific value or fall within a range of values

Eg: s_class CHAR(2) **CONSTRAINT** student_s_class_cc **CHECK**
((s_class = 'FR') **OR** (s_class = 'SO') **OR** (s_class = 'JR') **OR** (s_class = 'SR'))

- **NOT NULL constraints**: Specify whether a field value can be NULL
- **Unique constraints**: Specify that a field must have a unique value for every table record
- **Default constraints**: Specify that a field has a default value that the DBMS automatically inserts for every record, unless the user specifies an alternate value

Eg... (hire_date **DATE DEFAULT SYSDATE,**) ...

Table Creation With Constraints

```
CREATE TABLE faculty
(f_id NUMBER(6),
f_last VARCHAR2(30) NOT NULL,
f_first VARCHAR2(30),
loc_id NUMBER(6),
f_phone VARCHAR2(10),
f_rank VARCHAR2(8) CONSTRAINT fac_f_rank_cc CHECK ((f_rank =
'ASSO')
OR (f_rank = 'PROF') OR (f_rank = 'TUTO') OR (f_rank = 'LECT')),
f_sal NUMBER(8,2) NOT NULL DEFAULT 0,
F_email VARCHAR2(30) UNIQUE,
CONSTRAINT faculty_f_id_pk PRIMARY KEY (f_id),
CONSTRAINT faculty_loc_id_fk FOREIGN KEY (loc_id)
REFERENCES location(loc_id));
```

Understanding NULL Value

- Represents value for an attribute that is currently unknown or not applicable for tuple.
- Deals with incomplete or exceptional data.
- Represents the absence of a value and is not the same as zero or spaces, which are values.

Eg: **INSERT INTO** student (**StId**, **StLname**,
StFname, **StAdId**) values (**10090**,
'Cooper', **'Samantha'**, **NULL**)

Enforcing Referential Integrity Constraint

Consider Enroll and Student

- Student (sid, name, login, cgpa)
- Enrolled (sid, cid, grade)

What should be done if an Enrolled tuple with a non-existent student id is inserted?

- Create the non-existent Student ID at Student table.
- Pick the valid Student ID from Student table.

What should be done if a Student tuple is deleted/updated?

- The values from the Enrolled table referencing Student table should be updated/deleted as well.

Enforcing Referential Integrity Constraint

- ❑ Cascade delete
- ❑ Cascade update
- ❑ No action/restrict
- ❑ Set Default
- ❑ Set Null

Appropriate action depends on the tables involved.

Views

A view is a **virtual** or derived relation.

- **Base Relation**

Named relation corresponding to an entity in conceptual schema, whose tuples are **physically stored** in database.

- **View**

Dynamic result of one or more relational operations **operating on base relations** to produce another relation.

Base Relation and View

Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

*Base Relation
(All Data in Staff Table)*

CREATE VIEW Staff3

AS SELECT staffNo, fName, lName, position, sex
FROM Staff
WHERE branchNo = 'B003';

staffNo	fName	lName	position	sex
SG37	Ann	Beech	Assistant	F
SG14	David	Ford	Supervisor	M
SG5	Susan	Brand	Manager	F

View 1 (Vertical View)

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003

View 2 (Horizontal View)

CREATE VIEW Manager3Staff
AS SELECT *
FROM Staff
WHERE branchNo = 'B003';

Views

- A virtual relation that does not necessarily, actually exist in the database but is produced upon request, at time of request.
- Allows each user to have his or her own view of the database.
- A view is essentially some subset of the database.
- Contents of a view are defined as a query on one or more base relations.
- Views are dynamic, meaning that changes made to base relations that affect view attributes are immediately reflected in the view.

Simple View

```
CREATE VIEW faculty_view AS  
SELECT f_id, f_last, f_first, f_mi, loc_id, f_phone, f_rank  
FROM faculty;
```

Creating a view

```
CREATE [OR REPLACE] VIEW viewname [(alias,[alias]....)]  
AS subquery  
[WITH CHECK OPTION [CONSTRAINT constraint]]  
[WITH READ ONLY[CONSTRAINT constraint]]
```

Advantages and Disadvantages of Views

ADVANTAGES	DISADVANTAGES
Data Independence	Update restriction
Currency	Structure restriction
Improved Security	Performance
Reduced Complexity	
Convenience	
Customization	
Data Integrity	

The End of Lecture 2

Thank You